Pandas & Sklearn syntax primer

1. First look (EDA)

```
df.shape  # (rows, cols)
df.info()  # dtypes + non-null counts
df.describe(include='all')  # + unique/top/freq for text cols
df.head()
df.nunique()  # distinct values per column
df['col'].value_counts(dropna=False)
df['col'].value_counts(normalize=True)
# shares instead of counts
df.isna().sum()  # missing count
df.isna().mean()  # missing rate (0-1)
df.duplicated().sum()  # duplicate rows
df.select_dtypes('number').corr()['target'].sort_values()
```

```
# heatmap: correlations or any pivot table
import seaborn as sns
sns.heatmap(df.select_dtypes('number').corr(), annot=True, fmt='.2f',
            cmap='coolwarm', center=0)
# pandas only: color the table itself in the notebook
(df.select_dtypes('number').corr()
   .style.background_gradient(cmap='coolwarm'))
```

2. Filtering and selecting

```
# each condition in ( ): & | bind tighter than ==
df[(df['a'] == 'x') | (df['b'] > 5)]  # or
df[(df['a'] == 'x') & (df['b'] > 5)]  # and
df[~(df['a'] == 'x')]  # not
df[df['action'].isin(['page_load', 'page_exit'])]
df[df['price'].between(10, 20)]  # inclusive
df[df['name'].str.contains('abc', na=False)]
df.query("action == 'sent' and amount > 5")
df.loc[mask, ['col1', 'col2']]  # rows by mask, cols by name
df.nlargest(5, 'col')  # top 5 rows by col
```

3. Groupby and aggregation

```
df.groupby('g')['x'].mean().reset_index()
# reset_index -> normal DataFrame
df.groupby(['g1', 'g2'])['x'].agg(['mean', 'max', 'count'])
df.groupby('g').size()  # rows incl. NaN; .count() = non-null only
# named aggregation: clean column names in one step
df.groupby('g').agg(
    avg_price=('price', 'mean'),
    n_orders=('order_id', 'nunique'),
    last_seen=('timestamp', 'max'),
).reset_index()
# a rate = mean of a boolean
df['is_accepted'] = df['action_acc'].notna()
df.groupby('date')['is_accepted'].mean()
# keep row count, add group stat to every row
df['user_avg'] = df.groupby('user_id')['x'].transform('mean')
# rank within group
df['rank'] = df.groupby('g')['x'].rank(ascending=False, method='dense')
# wide table: one column per category
df.pivot_table(index='date', columns='action', values='id', aggfunc='count', fill_value=0)
```

```
df = df.sort_values("order_ts_utc")
df["is_first"] = df.groupby(["customer_id", "device_id"]).cumcount() == 0  # cumcount: 0,1,2.. per group in current order (sort first)
# last row instead: cumcount(ascending=False) == 0
```

```
# copy an aggregate back onto rows: map (1 col) / left merge (several)
s = orders_combined.groupby("order_id")["order_ts_utc_other"].count()
df["n_card_orders_window"] = df["order_id"].map(s).fillna(0).astype(int)  # keys absent from agg -> NaN
# several columns at once
agg = orders_combined.groupby("order_id").agg(n=("order_id_other", "count")).reset_index()
df = df.merge(agg, on="order_id", how="left")
```

4. Merging and combining

```
# merge on ALL keys defining a match; check len() before/after (jump = dup keys)
a.merge(b, on=['user_id', 'date'], how='inner')  # inner | left | right | outer
a.merge(b, left_on='uid', right_on='user_id')
a.merge(b, on='id', how='left', suffixes=('_sent', '_acc'))
# which rows matched?
m = a.merge(b, on='id', how='left', indicator=True)
m['_merge'].value_counts()  # both / left_only / right_only
# rows in a but not in b (anti-join)
m[m['_merge'] == 'left_only']
# stack rows / align Series side by side on index
pd.concat([df1, df2], ignore_index=True)
pd.concat([s1, s2], axis=1, keys=['load', 'exit'])
```

5. Dates

```
df['ts'] = pd.to_datetime(df['ts'], errors='coerce')
# bad values -> NaT
# Multiple formats, autoresolve
df["date"] = pd.to_datetime(df["date"], format="mixed")
df['date'] = df['ts'].dt.date  # python date objects
df['day'] = df['ts'].dt.normalize()  # keeps datetime dtype
df['month'] = df['ts'].dt.to_period('M')
df['year'], df['weekday'] = df['ts'].dt.year, df['ts'].dt.day_name()
df['hour'] = df['ts'].dt.hour
# differences
delta = df['end'] - df['start']  # timedelta
delta.dt.total_seconds()  # -> float seconds
delta.dt.days
# time-based filter / resample
df[df['ts'] >= '2024-01-01']
df.set_index('ts').resample('W')['sales'].sum()
```

Lags and rolling features (shift)

```
# shift counts ROWS, not time: sort by date per group + fill missing months first
s.shift(1)  # value from previous row:  [10,20,30] -> [NaN, 10,20]
s.shift(-1)  # value from NEXT row:      [10,20,30] -> [20,30,NaN]
s.diff(1)  # s - s.shift(1)
s.pct_change() # % change vs previous row
# per group: never leaks one category into another
df = df.sort_values(['category', 'month'])
g = df.groupby('category')['revenue']
df['lag_1'] = g.shift(1)
df['lag_12'] = g.shift(12)  # NaN for first 12 months (no data, not 0)
df['rolling_3'] = g.transform(lambda s: s.shift(1).rolling(3).mean())  # shift FIRST: no leakage
# first / last non-NaN (labels); positions: np.flatnonzero(s.notna())
s.first_valid_index(), s.last_valid_index()
# never lag via df.loc[a, 'lag'] = df.loc[b, 'revenue']: .loc aligns on LABELS -> use shift (or .to_numpy())
```

6. Cleaning

```
# missing values
df.dropna(subset=['col'])
df['col'] = df['col'].fillna(df['col'].median())
df['cat'] = df['cat'].fillna('missing')
df = df.loc[:, df.isna().mean() < 0.5]  # drop cols >50% missing
# duplicates
df.drop_duplicates()
# keep only the latest row per key (sort first, else 'last' = file order)
df = (df.sort_values('updated_at')
        .drop_duplicates(subset=['customer_id'], keep='last'))
# inspect every row in a duplicate group
df[df.duplicated(subset=['customer_id'], keep=False)]
```

```
# types
df['x'] = pd.to_numeric(df['x'], errors='coerce')
df['cat'] = df['cat'].astype('category')
# strings
df['name'] = df['name'].str.strip().str.lower()
df[['first', 'last']] = df['name'].str.split(' ', n=1, expand=True)
# rename, sort, round, map
df.rename(columns={'old': 'new'})
df.sort_values(['a', 'b'], ascending=[True, False])
df['x'].round(2)
df['flag'] = df['grade'].map({'A': 1, 'B': 0})
df['bucket'] = pd.cut(df['age'], bins=[0, 18, 65, 120], labels=['kid', 'adult', 'senior'])
```

```
# '$4,799', 'call for price' -> 4799.0, NaN
df['price'] = pd.to_numeric(df['price'].astype(str).str.replace(r'[$,]', '', regex=True), errors='coerce')
# '€29.99', '29,99', '29.99 EUR' (comma = decimal)
df['fee'] =
pd.to_numeric(df['fee'].str.replace(r'[^\d.,]', '', regex=True)
                       .str.replace(',', '.', regex=False), errors='coerce')
```

```
# one-hot: columns from TRAIN only, unseen test value -> all 0
tr = pd.get_dummies(X_train['fuel'], prefix='fuel', dtype=int)
te = pd.get_dummies(X_test['fuel'], prefix='fuel', dtype=int).reindex(columns=tr.columns, fill_value=0)
X_train = pd.concat([X_train.drop(columns='fuel'), tr], axis=1)
X_test = pd.concat([X_test.drop(columns='fuel'), te], axis=1)
# scale: fit on train, transform test
sc = StandardScaler()
X_train[cols] = sc.fit_transform(X_train[cols])
X_test[cols] = sc.transform(X_test[cols])
```

7. sklearn pipeline template

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
X = df.drop(columns=['target', 'id'])
y = df['target']
num_cols = X.select_dtypes(include='number').columns
cat_cols = X.select_dtypes(exclude='number').columns
preprocess = ColumnTransformer([
    ('num', Pipeline([('impute', SimpleImputer(strategy='median')),
                      ('scale', StandardScaler())]), num_cols),
    ('cat', Pipeline([('impute', SimpleImputer(strategy='most_frequent')),
                      ('onehot', OneHotEncoder(handle_unknown='ignore'))]), cat_cols),
], remainder='passthrough')  # default 'drop': columns not listed are silently dropped
# preprocess.set_output(transform='pandas')  -> keyword only; needs OneHotEncoder(sparse_output=False)
model = Pipeline([('prep', preprocess),
                  ('clf', RandomForestClassifier(random_state=42))])  # class_weight='balanced' if imbalanced
# swap: LogisticRegression, DecisionTreeClassifier(max_depth=5), LinearRegression, LassoCV, RandomForestRegressor
# order: X_train, X_test, y_train, y_test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)  # stratify: classification only
model.fit(X_train, y_train)
preds = model.predict(X_test)
```

```
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')  # full X, y: refit per fold, no leakage
scores.mean(), scores.std()  # average score + how much it varies across folds
# scoring options
# classification: 'accuracy', 'f1', 'roc_auc', 'precision', 'recall'
# regression:     'r2', 'neg_mean_absolute_error', 'neg_root_mean_squared_error'
# ('neg_' scores are negated errors: closer to 0 is better)
```

```
from sklearn.model_selection import GridSearchCV
# grid search on train only; params = <step>__<param>
grid = {'clf__n_estimators': [100, 300], 'clf__max_depth': [None, 5, 10]}
gs = GridSearchCV(model, grid, cv=5, scoring='f1', n_jobs=-1).fit(X_train, y_train)
gs.best_params_, gs.best_score_
gs.best_estimator_.score(X_test, y_test)  # already refit on all of X_train
# model.get_params().keys() lists valid names
```

```
# evaluate
from sklearn.metrics import (accuracy_score, f1_score, roc_auc_score,
                             classification_report, confusion_matrix,
                             mean_absolute_error, mean_squared_error, r2_score)
# classification
classification_report(y_test, preds)
confusion_matrix(y_test, preds)
roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
# regression
mean_absolute_error(y_test, preds)
mean_squared_error(y_test, preds) ** 0.5  # RMSE
r2_score(y_test, preds)
```

Threshold for a precision target

```
# precision >= target, flag as many as possible: LOWEST threshold that clears it
from sklearn.metrics import precision_recall_curve
import numpy as np
prec, rec, thr = precision_recall_curve(y_val, y_val_pred_probs)
# prec/rec have one more element than thr; drop the last to align
prec, rec = prec[:-1], rec[:-1]
target = 0.85  # aim above 0.80: test precision will wobble
ok = prec >= target
best = thr[ok].min()  # lowest threshold = most people flagged
print(best, prec[thr == best], rec[thr == best], (y_val_pred_probs >= best).mean())
test_probs = pipeline.predict_proba(X_test)[:, 1]
churned = (test_probs >= best).astype(int)
churned.mean()  # share flagged: check minimum (e.g. ≥ 2%)
```

```
# probs must beat base rate; else CalibratedClassifierCV(model, cv=5) or LogisticRegression, then redo threshold
from sklearn.metrics import brier_score_loss
brier_score_loss(y_val, y_val_pred_probs)  # must be lower than:
brier_score_loss(y_val, np.full(len(y_val), y_train.mean()))
```

8. Prediction file template

```
# fit on ALL of train; output exactly the columns + dtypes asked
train = pd.read_csv('train.csv')
test = pd.read_csv('test.csv')
X, y = train.drop(columns=['target', 'id']), train['target']
model.fit(X, y)
preds = model.predict(test.drop(columns=['id']))
# probabilities instead: model.predict_proba(...)[:, 1]
out = pd.DataFrame({'id': test['id'], 'target': preds})
out.to_csv('predictions.csv', index=False)
pd.read_csv('predictions.csv').head()  # check before submitting
```